EXPT NO: 4	A PYTHON PROGRAM TO PERFORM PERT ANALYSIS
DATE:	

AIM:

To solve any integer programming problem using the Branch and Bound method in Python.

PROCEDURE:

1. Formulate the integer programming problem.
2. Implement the Branch and Bound method in Python.
3. Solve the problem using the implemented method.
4. Verify the solution.

CODE

PROGRAM 1:

```
import numpy as np
from scipy.optimize import linprog
from queue import Queue
import copy

# Solve LP relaxation
def solve_lp(c, A, b, bounds):
    res = linprog(c, A_ub=A, b_ub=b, bounds=bounds, method='highs')
    return res

# Branch and Bound method
def branch_and_bound(c, A, b, bounds):
    Q = Queue()
    Q.put((c, A, b, bounds))

    best_solution = None
    best_value = float('-inf')
```

```
    while not Q.empty():
        current_problem = Q.get()
        res = solve_lp(*current_problem)

        if not res.success:
            continue

        # Since we minimized negative, convert back
        if -res.fun > best_value:
            solution = res.x

            # Check if integer solution
            if all(np.isclose(solution, np.round(solution))):
                value = -res.fun
                if value > best_value:
                    best_value = value
                    best_solution = solution
            else:
                # Branch on first fractional variable
                for i in range(len(solution)):
                    if not np.isclose(solution[i], np.round(solution[i])):

                        lower_bound = copy.deepcopy(current_problem[3])
                        upper_bound = copy.deepcopy(current_problem[3])

                        # x_i <= floor(value)
                        lower_bound[i] = (lower_bound[i][0], np.floor(solution[i]))

                        # x_i >= ceil(value)
                        upper_bound[i] = (np.ceil(solution[i]), upper_bound[i][1])

                        # Add valid branches
                        if lower_bound[i][1] is None or lower_bound[i][0] <= lower_bound[i][1]:
                            Q.put((c, A, b, lower_bound))

                        if upper_bound[i][1] is None or upper_bound[i][0] <= upper_bound[i][1]:
                            Q.put((c, A, b, upper_bound))

                        break

    return best_solution, best_value


# Problem definition
c = [-4, -3]  # Maximize 4x + 3y
```

```
A = [
    [2, 1],
    [1, 2]
]
b = [8, 6]

# Variable bounds (x >= 0, y >= 0)
bounds = [(0, None), (0, None)]

# Solve
solution, value = branch_and_bound(c, A, b, bounds)

# Output
print("Optimal solution:", solution)
print("Optimal value:", value)
```

OUTPUT:

Optimal solution: [3. 1.]
Optimal value: 15.0

PROGRAM 2:

```
import numpy as np
from scipy.optimize import linprog
from queue import Queue
import copy

# Solve LP relaxation
def solve_lp(c, A, b, bounds):
    res = linprog(c, A_ub=A, b_ub=b, bounds=bounds, method='highs')
    return res

# Branch and Bound
def branch_and_bound(c, A, b, bounds):
    Q = Queue()
    Q.put((c, A, b, bounds))

    best_solution = None
    best_value = float('-inf')

    while not Q.empty():
        current_problem = Q.get()
        res = solve_lp(*current_problem)
```

```
        if not res.success:
            continue

        # Convert back from minimization
        if -res.fun > best_value:
            solution = res.x

            # Check integer solution
            if all(np.isclose(solution, np.round(solution))):
                value = -res.fun
                if value > best_value:
                    best_value = value
                    best_solution = solution
            else:
                # Branch
                for i in range(len(solution)):
                    if not np.isclose(solution[i], np.round(solution[i])):

                        lower_bound = copy.deepcopy(current_problem[3])
                        upper_bound = copy.deepcopy(current_problem[3])

                        # x_i <= floor
                        lower_bound[i] = (lower_bound[i][0], np.floor(solution[i]))

                        # x_i >= ceil
                        upper_bound[i] = (np.ceil(solution[i]), upper_bound[i][1])

                        if lower_bound[i][1] is None or lower_bound[i][0] <= lower_bound[i][1]:
                            Q.put((c, A, b, lower_bound))

                        if upper_bound[i][1] is None or upper_bound[i][0] <= upper_bound[i][1]:
                            Q.put((c, A, b, upper_bound))

                        break

    return best_solution, best_value


# Problem definition
c = [-5, -2]  # Maximize 5x + 2y
A = [
    [1, 2],
    [3, 1]
]
b = [6, 9]
```

```
# Bounds
bounds = [(0, None), (0, None)]

# Solve
solution, value = branch_and_bound(c, A, b, bounds)

# Output
print("Optimal solution:", solution)
print("Optimal value:", value)
```

OUTPUT:

Optimal solution: [2. 2.]
Optimal value: 14.0

RESULT:

The Assignment Problem Has been Completed successfully.